AN ASSIGNMENT ON STRUCTURED PROGRAMMING LANGUAGE

Submitted To:

Name	**:**	Ditee Yasmeen
Course Title	**:**	Structured Programming Language

Submitted By:

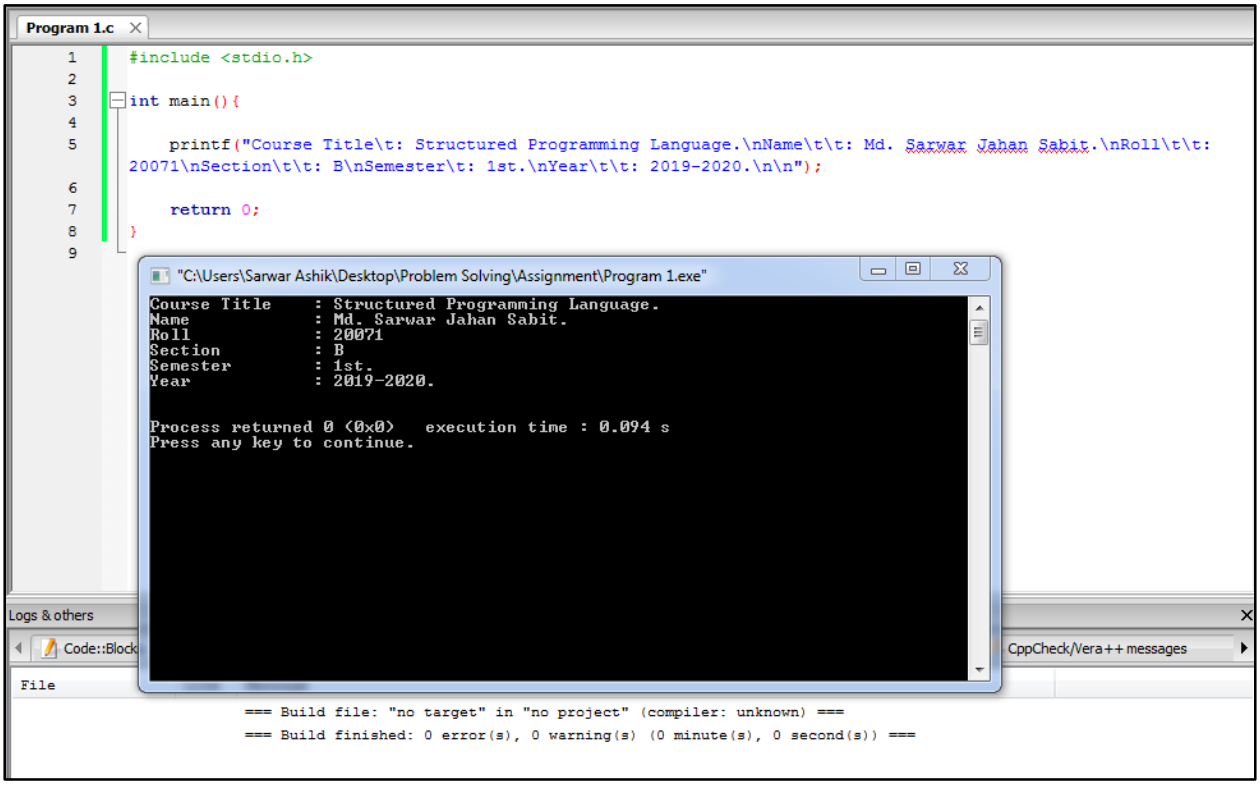
Name	**:**	Md. Sarwar Jahan Sabit
Department	**:**	CSE
Roll	**:**	20071
Section	**:**	B
Semester	**:**	1st
Session	**:**	2019-2020

1. Print your Course Title, Name, Roll, Section, Semester and Year.

Here is a program in C that prints out my course title, name, roll, section, semester and year.

```c
#include <stdio.h>

int main(){

    printf("Course Title\t: Structured Programming Language.\nName\t\t: Md. Sarwar Jahan Sabit.\nRoll\t\t: 20071\nSection\t\t: B\nSemester\t: 1st.\nYear\t\t: 2019-2020.\n\n");

    return 0;
}
```

Program 1: Printing Course Title, Name, Roll, Section, Semester and Year



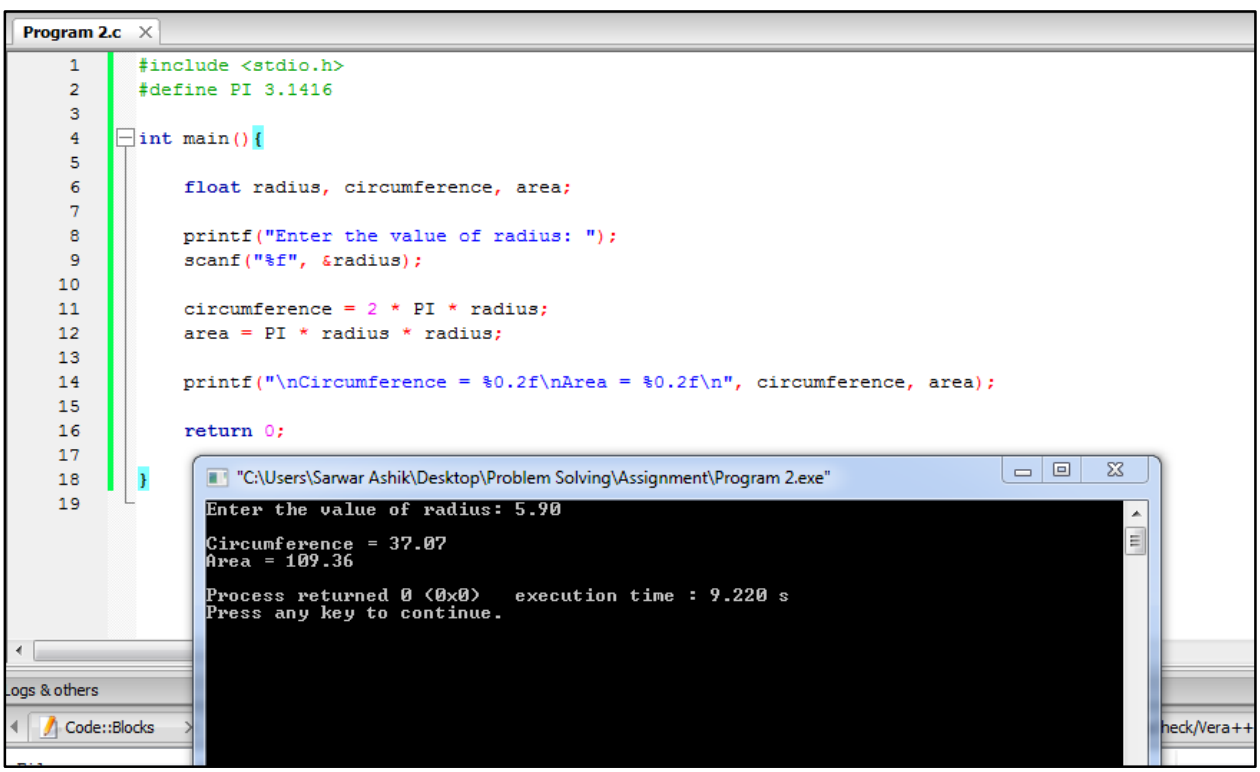
Screenshot 1: Running Program 1 in CodeBlocks and the output.

2. Find area and circumference of circle.

Here is a program in C that takes the value of diameter as input and prints out the circumference of a circle.

```c
#include <stdio.h>
#define PI 3.1416

int main(){

    float radius, circumference, area;

    printf("Enter the value of radius: ");
    scanf("%f", &radius);

    circumference = 2 * PI * radius;
    area = PI * radius * radius;

    printf("\nCircumference = %0.2f\nArea = %0.2f\n", circumference, area);

    return 0;

}
```

Program 2: Finding area and circumference of circle.



Screenshot 2: Running Program 2 in CodeBlocks and the output.

3. Find maximum and minimum among three numbers.

Here is a program in C that takes 3 integer values as input and prints out the maximum and minimum value.

```
#include <stdio.h>
#define PI 3.1416

int main(){

    int num1, num2, num3, max, min;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &num1, &num2, &num3);

    if(num1 >= num2 && num1>= num3){
        max = num1;
    }
    else if(num2 >= num1&& num2 >= num3){
        max = num2;
    }
    else{
        max = num3;
    }


    if(num1<= num2 && num1<= num3){
        min = num1;
    }
    else if(num2 <= num1&& num2 <= num3){
        min = num2;
    }
    else{
        min = num3;
    }

    printf("\nMaximum: %d\nMinimum: %d\n", max, min);

    return 0;

}
```

Program 3: Finding maximum and minimum among three numbers.

Program 3.c

```c
#include <stdio.h>
#define PI 3.1416

int main(){

    int num1, num2, num3, max, min;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &num1, &num2, &num3);

    if(num1 >= num2 && num1>= num3){
        max = num1;
    }
    else if(num2 >= num1&& num2 >= num3){
        max = num2;
    }
    else{
        max = num3;
    }


    if(num1<= num2 && num1<= num3){
        min = num1;
    }
    else if(num2 <= num1&& num2 <= num3){
```

Logs & others

Screenshot 3.1: Running Program 3 in CodeBlocks.

Program 3.c

```c
    else if(num2 >= num1&& num2 >= num3){
        max = num2;
    }
    else{
        max = num3;
    }


    if(num1<= num2 && num1<= num3){
        min = num1;
    }
    else if(num2 <= num1&& num2 <= num3){
        min = num2;
    }
    else{
        min = num3;
    }

    printf("\nMaximum: %d\nMinimum: %d\n", max, min);

    return 0;

}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assig...

```
Enter three numbers: 43 90 78

Maximum: 90
Minimum: 43

Process returned 0 (0x0)   execution time : 15.709 s
Press any key to continue.
```

Logs & others

Code::Blocks | Search results | Cccc | Build log | Build messages | CppCheck/Vera++ | CppCheck/Vera++ messages

File	Line	Message
		=== Build file: "no target" in "no project" (compiler: unknown) ===
		=== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===

Screenshot 3.2: Running Program 3 in CodeBlocks and the output.

4. Find all possible roots of $ax^2 + bx + c = 0$.

Here is a program in C that takes 3 integer values (a, b, c) as input and prints out all the values of x.

```
#include <stdio.h>
#include <math.h>

int main(){

    float a, b, c, d, r1, r2;

    input:
    printf("Enter the value of a, b and c: ");
    scanf("%f %f %f", &a, &b, &c);

    if(a == 0){
        printf("\nInvalid input. Value of a can't be 0. Please enter the
values again!\n");
        goto input;
    }
    else{
        d = b * b - 4 * a *c;

        if(d == 0){
            printf("\nOne value of x is found.\nx = %f\n", -
b/(float)(2*a));
        }
        else if(d > 0){
            printf("\nTwo value of x is found.\nx1 = %f\nx2 = %f\n", (-b +
sqrt(d)) /(float)(2*a), (-b - sqrt(d)) /(float)(2*a));
        }
        else if(d < 0){
            d = -d;
            r1 = -b / (float)(2*a);
            r2 = sqrt(d) / (float)(2*a);
            printf("\nImaginary value of x is found.\nThey are: %f + i(%f)
and %f - i(%f)", r1, r2, r1, r2);
        }
    }

    return 0;

}
```

Program 4: Finding all possible roots of ax2 + bx + c = 0.

Program 4.c

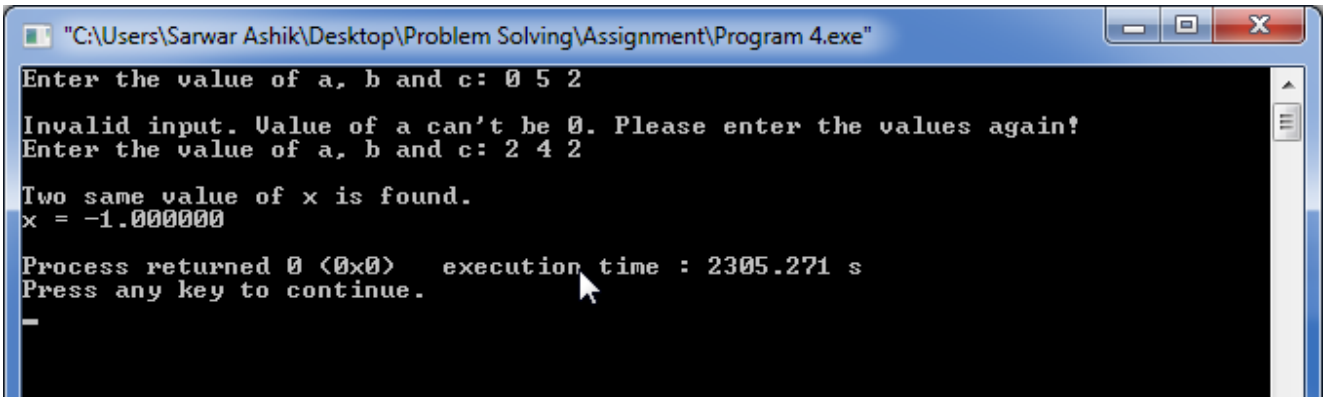
```c
#include <stdio.h>
#include <math.h>

int main(){

    float a, b, c, d, r1, r2;

    input:
    printf("Enter the value of a, b and c: ");
    scanf("%f %f %f", &a, &b, &c);

    if(a == 0){
        printf("\nInvalid input. Value of a can't be 0. Please enter the values again!\n");
        goto input;
    }
    else{

        d = b * b - 4 * a *c;

        if(d == 0){
            printf("\nOne value of x is found.\nx = %f\n", -b/(float)(2*a));
        }
        else if(d > 0){
            printf("\nTwo value of x is found.\nx1 = %f\nx2 = %f\n", (-b + sqrt(d)) /(float)(2*a), (-b - sqrt(d))
/(float)(2*a));
        }
```

Screenshot 4.1: Running Program 4 in CodeBlocks.

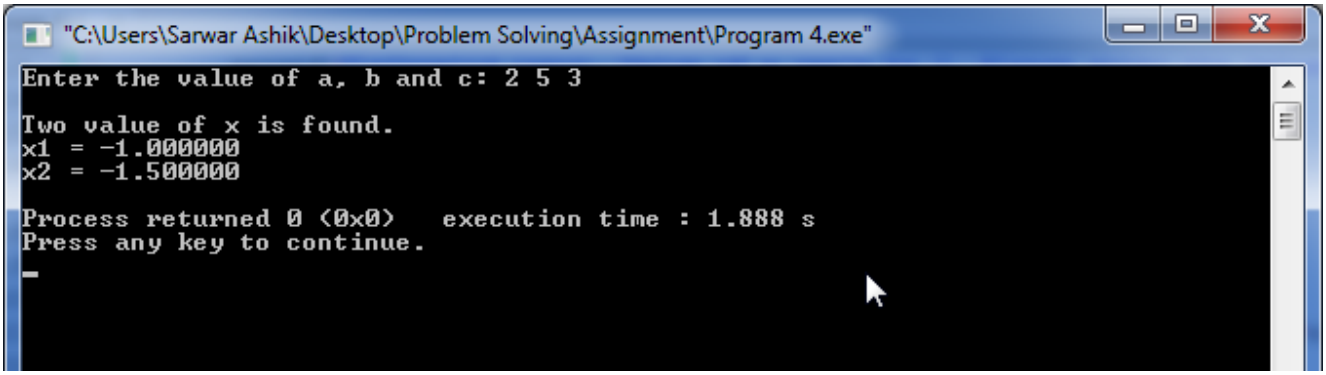
Program 4.c

```c
        printf("\nInvalid input. Value of a can't be 0. Please enter the values again!\n");
        goto input;
    }
    else{

        d = b * b - 4 * a *c;

        if(d == 0){
            printf("\nTwo same value of x is found.\nx = %f\n", -b/(float)(2*a));
        }
        else if(d > 0){
            printf("\nTwo value of x is found.\nx1 = %f\nx2 = %f\n", (-b + sqrt(d)) /(float)(2*a), (-b - sqrt(d))
/(float)(2*a));
        }
        else if(d < 0){
            d = -d;
            r1 = -b / (float)(2*a);
            r2 = sqrt(d) / (float)(2*a);
            printf("\nImaginary value of x is found.\nThey are: %f + i(%f) and %f - i(%f)\n", r1, r2, r1, r2);
        }
    }

    return 0;

}
```

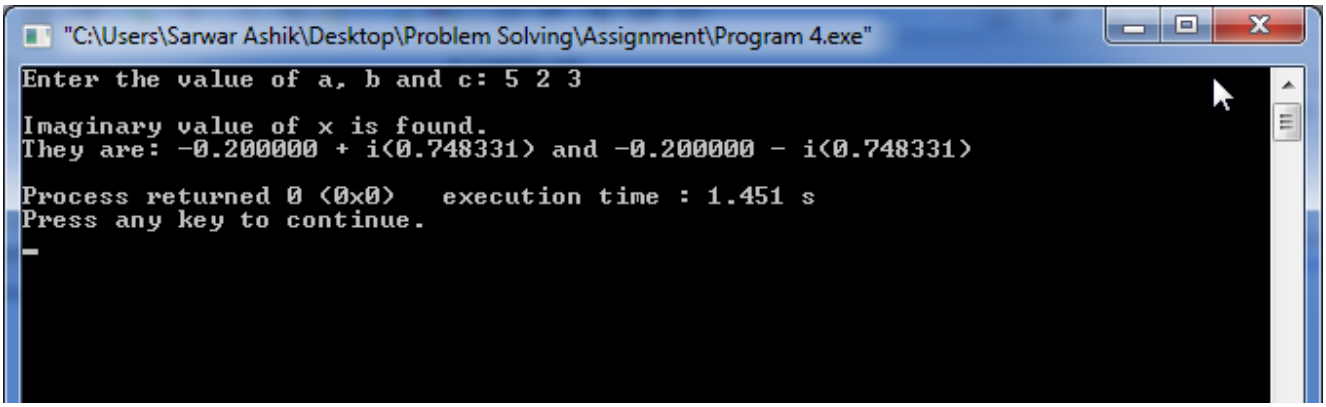
Screenshot 4.2: Running Program 4 in CodeBlocks.



Screenshot 4.3: Output of Program 4 (A).



Screenshot 4.4: Output of Program 4 (B).



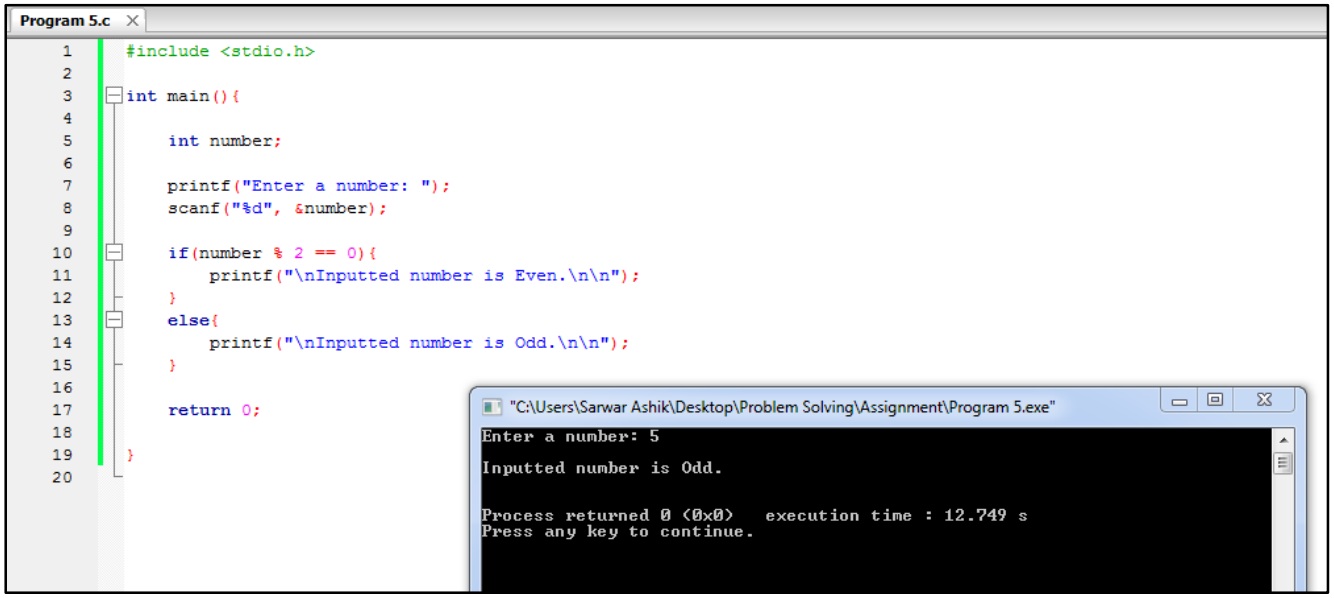
Screenshot 4.5: Output of Program 4 (C).

5. Verify whether the given number is odd or even?

Here is a program in C that takes a single integer value as input and prints out if the number is odd or even.

```c
#include <stdio.h>

int main(){

    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if(number % 2 == 0){
        printf("\nInputted number is Even.\n\n");
    }
    else{
        printf("\nInputted number is Odd.\n\n");
    }

    return 0;

}
```

Program 5: Verifying whether the given number is odd or even.



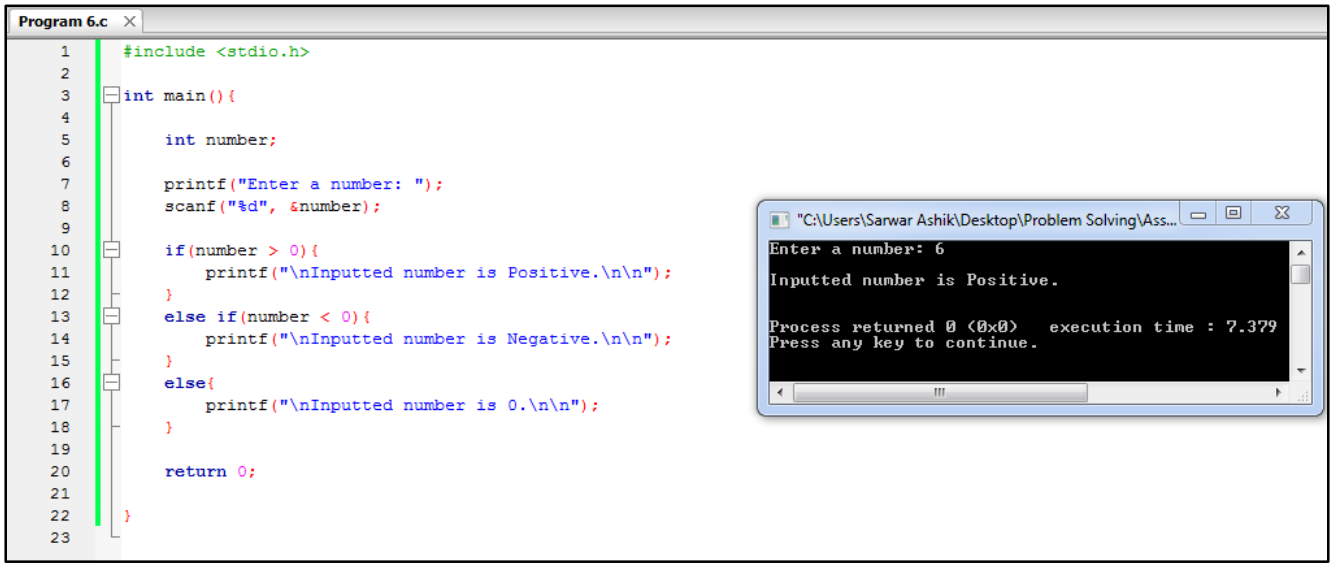
Screenshot 5: Running Program 5 in CodeBlocks and the output.

6. Verify whether the given number is positive or negative.

Here is a program in C that takes a single integer value as input and prints out if the number is positive or negative.

```c
#include <stdio.h>

int main(){

    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if(number > 0){
        printf("\nInputted number is Positive.\n\n");
    }
    else if(number < 0){
        printf("\nInputted number is Negative.\n\n");
    }
    else{
        printf("\nInputted number is 0.\n\n");
    }

    return 0;

}
```

Program 6: Verifying whether the given number is positive or negative.



Screenshot 6.1: Running Program 6 in CodeBlocks and the output.

Program 6.c

```c
#include <stdio.h>

int main(){

    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if(number > 0){
        printf("\nInputted number is Positive.\n\n");
    }
    else if(number < 0){
        printf("\nInputted number is Negative.\n\n");
    }
    else{
        printf("\nInputted number is 0.\n\n");
    }

    return 0;

}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Ass...

```
Enter a number: -10

Inputted number is Negative.


Process returned 0 (0x0)   execution time : 17.746
Press any key to continue.
```

Screenshot 6.2: Running Program 6 in CodeBlocks and the output.

Program 6.c

```c
#include <stdio.h>

int main(){

    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if(number > 0){
        printf("\nInputted number is Positive.\n\n");
    }
    else if(number < 0){
        printf("\nInputted number is Negative.\n\n");
    }
    else{
        printf("\nInputted number is 0.\n\n");
    }

    return 0;

}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assignmen...

```
Enter a number: 0

Inputted number is 0.


Process returned 0 (0x0)   execution time : 5.877 s
Press any key to continue.
```

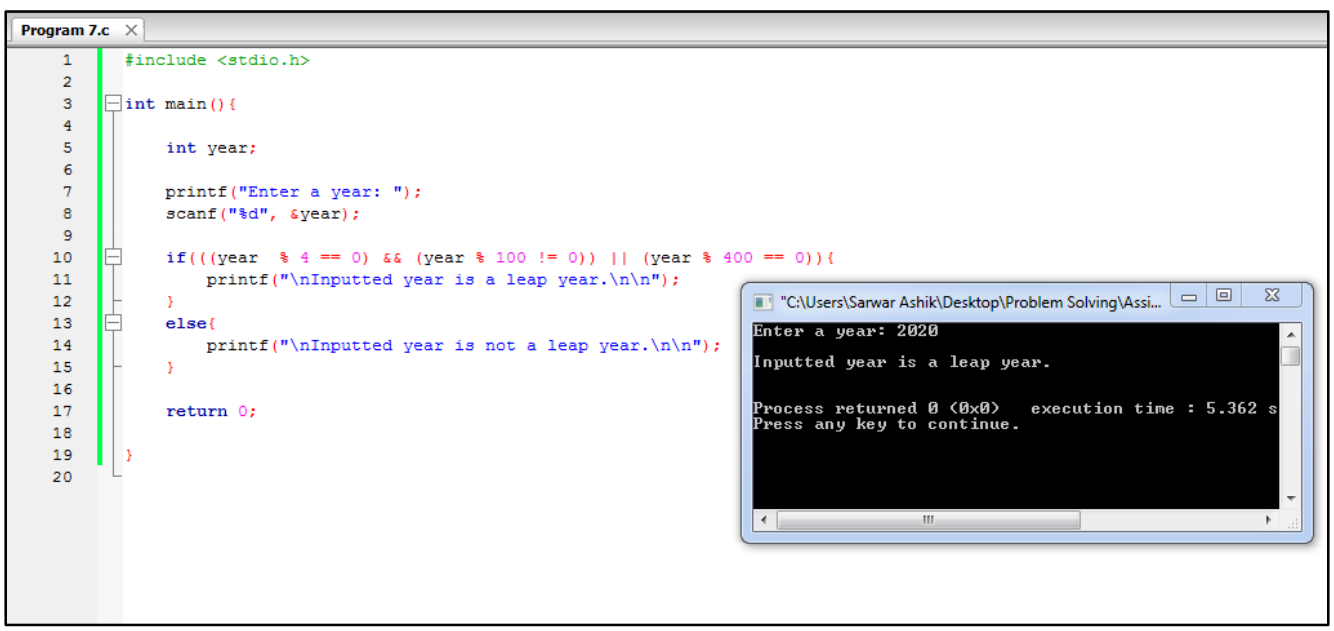
Screenshot 6.3: Running Program 6 in CodeBlocks and the output.

7. Verify whether the given year is leap year or not.

Here is a program in C that takes a single integer value as input for year and prints out if the year is a leap year or not.

```c
#include <stdio.h>

int main(){

    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if(((year  % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0)){
        printf("\nInputted year is a leap year.\n\n");
    }
    else{
        printf("\nInputted year is not a leap year.\n\n");
    }

    return 0;

}
```

Program 7: Verifying whether the given year is leap year or not.



Screenshot 7: Running Program 7 in CodeBlocks and the output.

8. Find maximum and minimum for n numbers in an array.

Here is a program in C that takes integers as input for an array of length n and prints out the minimum and maximum number.

```
#include <stdio.h>

int main(){

    int n, max, min, i;

    printf("Enter the length of array: ");
    scanf("%d", &n);
    printf("Enter the values of array: ");
    int nums_arr[n];

    for(i = 0; i < n; i++){
        scanf("%d", &nums_arr[i]);
    }

    max = nums_arr[0];
    min = nums_arr[0];

    for(i = 1; i < n; i++){
        if(max < nums_arr[i]){
            max = nums_arr[i];
        }

        if(min > nums_arr[i]){
            min = nums_arr[i];
        }
    }

    printf("\nMaximum Value: %d.\nMinimum Value: %d\n\n", max, min);

    return 0;

}
```

Program 8: Finding maximum and minimum for n numbers in an array.

Program 8.c

```c
#include <stdio.h>

int main(){

    int n, max, min, i;

    printf("Enter the length of array: ");
    scanf("%d", &n);
    printf("Enter the values of array: ");
    int nums_arr[n];

    for(i = 0; i < n; i++){
        scanf("%d", &nums_arr[i]);
    }

    max = nums_arr[0];
    min = nums_arr[0];

    for(i = 1; i < n; i++){
        if(max < nums_arr[i]){
            max = nums_arr[i];
        }

        if(min > nums_arr[i]){
            min = nums_arr[i];
        }
    }

    printf("\nMaximum Value: %d.\nMinimum Value: %d.\n\n", max, min);

    return 0;

}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assignment\Progr...

```
Enter the length of array: 5
Enter the values of array: 35 66 23 77 45

Maximum Value: 77.
Minimum Value: 23.


Process returned 0 (0x0)   execution time : 13.348 s
Press any key to continue.
```

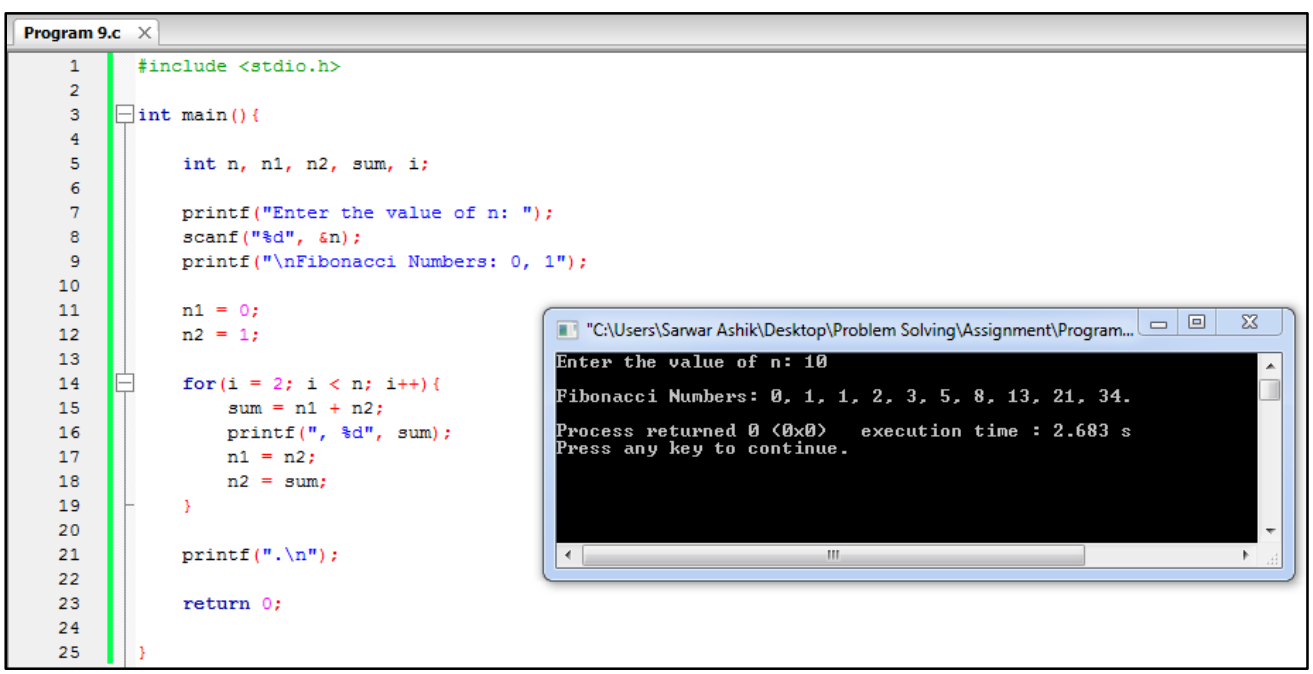
Screenshot 8: Running Program 8 in CodeBlocks and the output.

9. Print n Fibonacci numbers using iterative approach.

Here is a program in C that takes an integer as input for the value of n and prints out n Fibonacci numbers.

```c
#include <stdio.h>

int main(){

    int n, n1, n2, sum, i;

    printf("Enter the value of n: ");
    scanf("%d", &n);
    printf("\nFibonacci Numbers: 0, 1");

    n1 = 0;
    n2 = 1;

    for(i = 2; i < n; i++){
        sum = n1 + n2;
        printf(", %d", sum);
        n1 = n2;
        n2 = sum;
    }

    printf(".\n");

    return 0;

}
```

Program 9: Printing n Fibonacci numbers using iterative approach.



Screenshot 9: Running Program 9 in CodeBlocks and the output.

10. Find factorial of m where m > 10.

Here is a program in C that takes an integer as input to find out its factorial (may not work when m > 500).

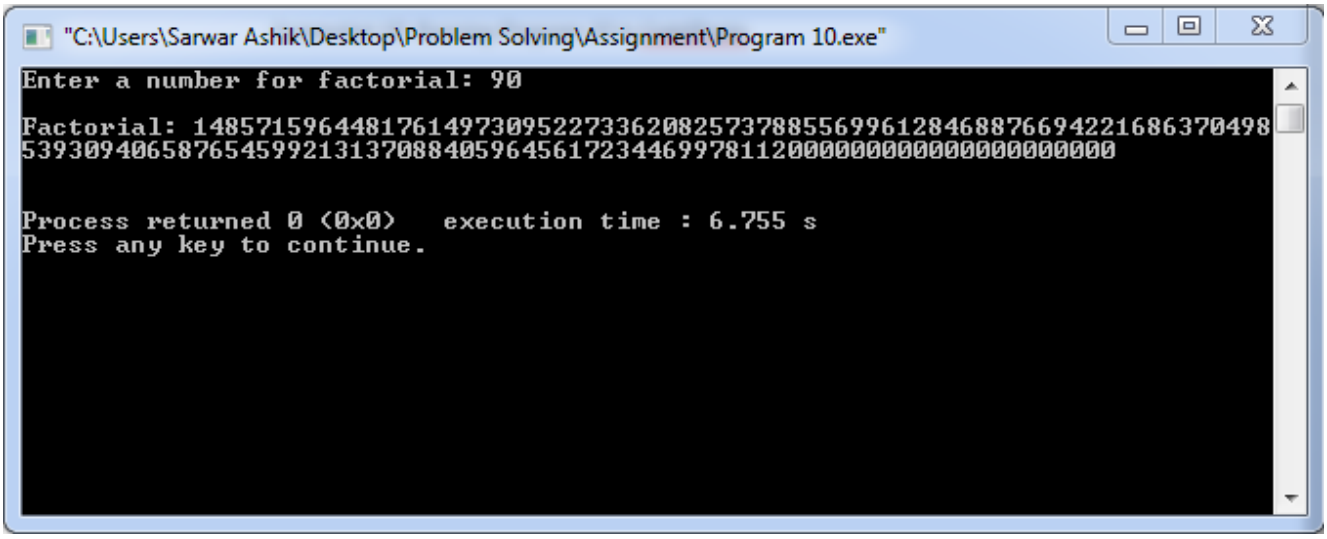
```
#include <stdio.h>

int main(){

    int arr[1000] = { 0 }, n, carry, i, j, temp, arr_len;

    printf("Enter a number for factorial: ");
    scanf("%d", &n);

      arr[0] = 1;
      arr_len = 1;

      carry = 0;
      for(i = 1; i <= n; i++) {
            for(j = 0; j < arr_len; j++) {
                  temp = arr[j] * i + carry;
                  arr[j] = temp % 10;
                  carry = temp / 10;
            }
            while(carry>0) {
                  arr[arr_len] = carry % 10;
                  carry = carry / 10;
                  arr_len++;
            }
      }

    printf("\nFactorial: ");
      for(i = arr_len - 1; i >= 0; i--){
            printf("%d", arr[i]);
      }

    printf("\n\n");

      return 0;
}
```

Program 100: Find factorial of m where m > 10.

Program 10.c

```c
#include <stdio.h>

int main(){

    int arr[1000] = { 0 }, n, carry, i, j, temp, arr_len;

    printf("Enter a number for factorial: ");\
    scanf("%d", &n);

    arr[0] = 1;
    arr_len = 1;

    carry = 0;
    for(i = 1; i <= n; i++) {
        for(j = 0; j < arr_len; j++) {
            temp = arr[j] * i + carry;
            arr[j] = temp % 10;
            carry = temp / 10;
        }
        while(carry>0) {
            arr[arr_len] = carry % 10;
            carry = carry / 10;
            arr_len++;
        }
    }

    printf("\nFactorial: ");
    for(i = arr_len - 1; i >= 0; i--){
        printf("%d", arr[i]);
    }

    printf("\n\n");

    return 0;
}
```

Screenshot 10.1: Running Program 10 in CodeBlocks.



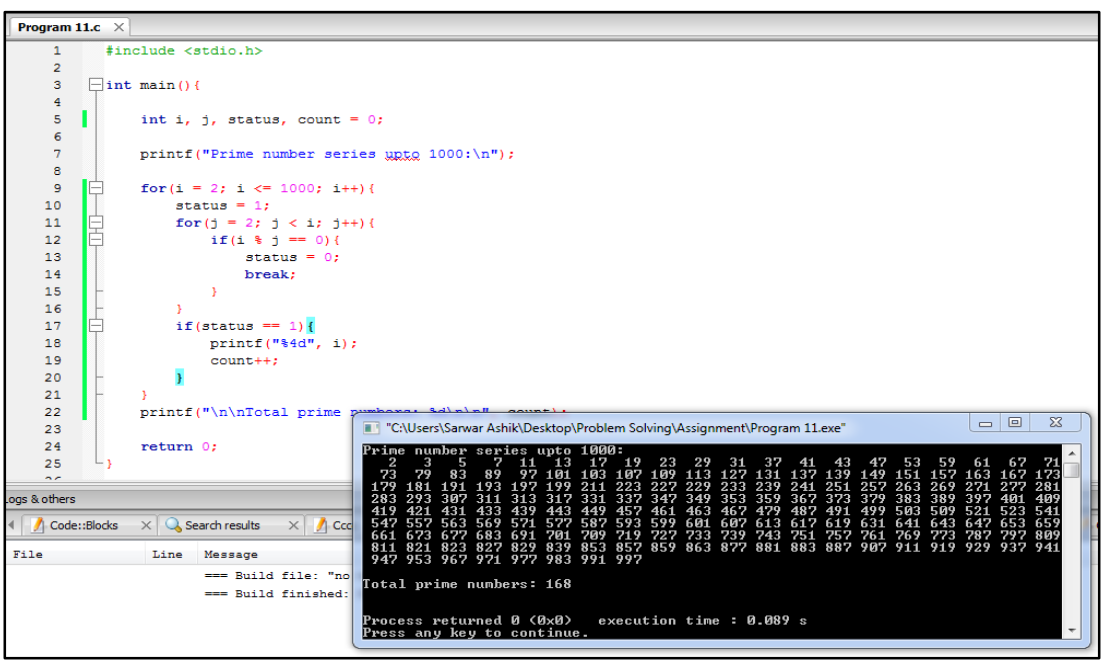
Screenshot 10.2: Output of program 10.

11. Find the series of all possible prime numbers up to value 1000.

Here is a program in C that prints out prime number series up to value 1000.

```c
#include <stdio.h>

int main(){

    int i, j, status, count = 0;

    printf("Prime number series upto 1000:\n");

    for(i = 2; i <= 1000; i++){
        status = 1;
        for(j = 2; j < i; j++){
            if(i % j == 0){
                status = 0;
                break;
            }
        }
        if(status == 1){
            printf("%4d", i);
            count++;
        }
    }
    printf("\n\nTotal prime numbers: %d\n\n", count);

    return 0;
}
```

Program 11: Finding the series of all possible prime numbers up to value 1000.



Screenshot 11: Running Program 11 in CodeBlocks and output.

12. Check whether the given number is Armstrong or not.

Here is a program in C that takes an integer as input and prints out whether the given number is Armstrong or not.

```
#include <stdio.h>
#include <math.h>

int main(){

    int number, tempnum, n = 0, remainder, sum = 0, i;

    printf("Enter a number to check whether the number is Armstrong or not: ");
    scanf("%d", &number);

    tempnum = number;

    while(tempnum != 0){
        tempnum /= 10;
        n++;
    }

    tempnum = number;

    while(tempnum != 0){
        remainder = tempnum % 10;
        tempnum /= 10;
        sum += pow(remainder, n);
    }

    if(number == sum){
        printf("\nInputted number is an Armstrong number.\n\n");
    }
    else{
        printf("\nInputted number isn't an Armstrong number.\n\n");
    }

    return 0;
}
```

Program 12: Checking whether the given number is Armstrong or not.

Program 12.c

```c
#include <stdio.h>
#include <math.h>

int main(){

    int number, tempnum, n = 0, remainder, sum = 0, i;

    printf("Enter a number to check whether the number is Armstrong or not: ");
    scanf("%d", &number);

    tempnum = number;

    while(tempnum != 0){
        tempnum /= 10;
        n++;
    }

    tempnum = number;

    while(tempnum != 0){
        remainder = tempnum % 10;
        tempnum /= 10;
        sum += pow(remainder, n);
    }

    if(number == sum){
        printf("\nInputted number is an Armstrong number.\n\n");
    }
    else{
        printf("\nInputted number isn't an Armstrong number.\n\n");
    }

    return 0;
}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assignment\Program 12.exe"

```
Enter a number to check whether the number is Armstrong or not: 1634

Inputted number is an Armstrong number.


Process returned 0 (0x0)   execution time : 13.385 s
Press any key to continue.
```

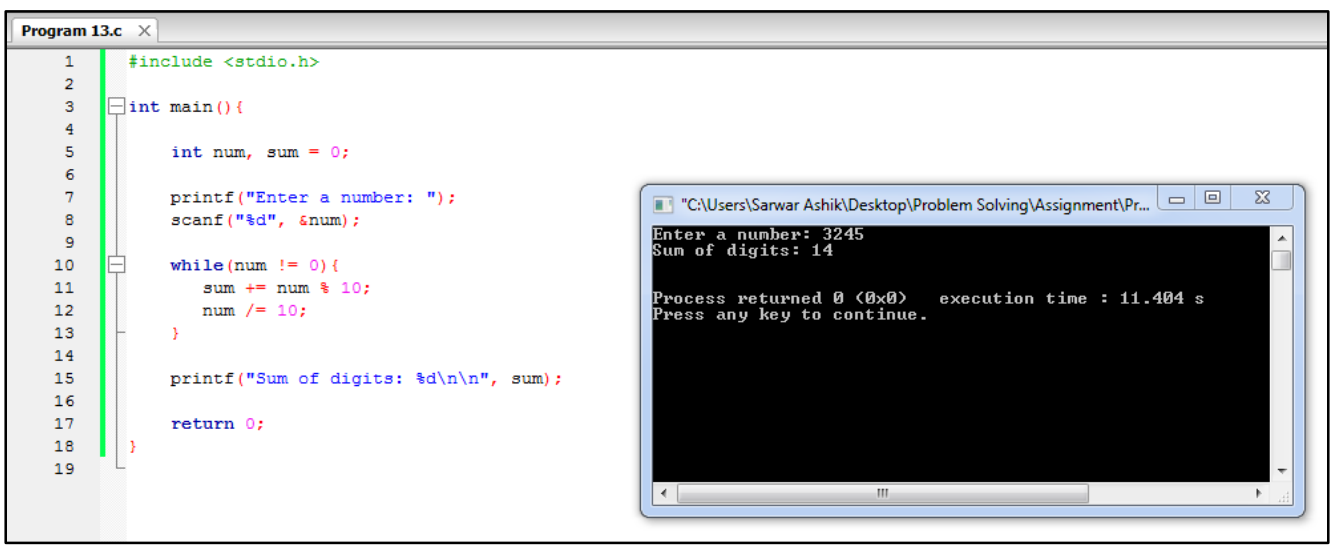
Screenshot 12: Running Program 12 in CodeBlocks and the output.

13. Print sum of digits of any inputted number.

Here is a program in C that takes an integer as input and prints out the sum of its digits.

```c
#include <stdio.h>

int main(){

    int num, sum = 0;

    printf("Enter a number: ");
    scanf("%d", &num);

    while(num != 0){
        sum += num % 10;
        num /= 10;
    }

    printf("Sum of digits: %d\n\n", sum);

    return 0;
}
```

Program 13: Printing sum of digits of any inputted number.



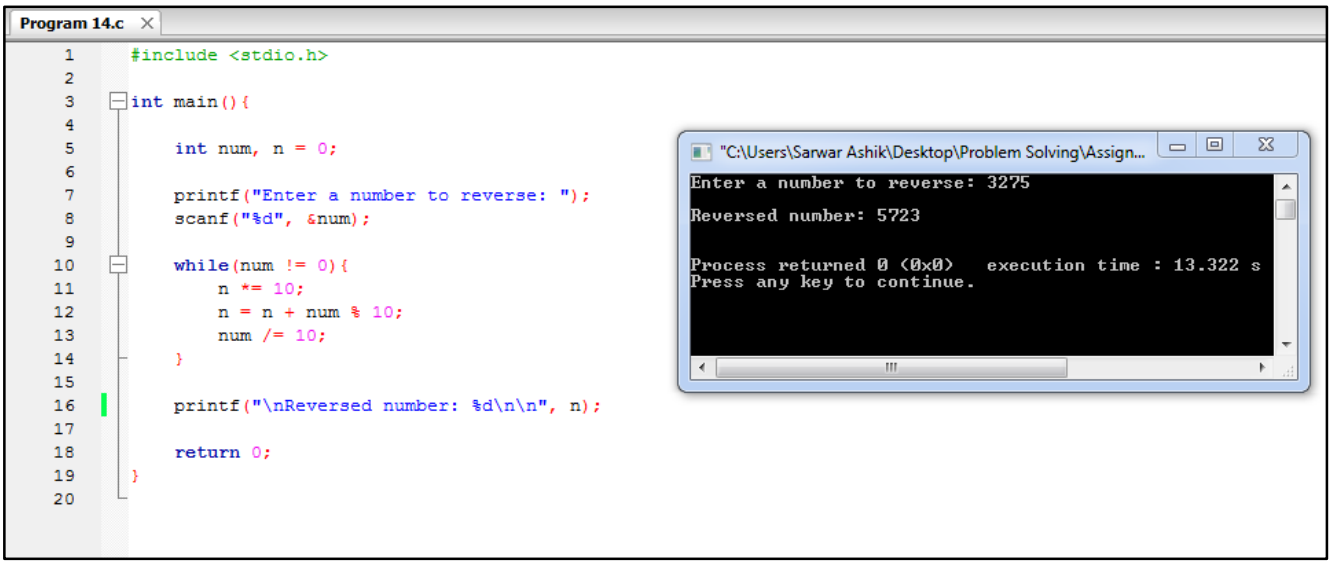
Screenshot 13: Running Program 13 in CodeBlocks and the output.

14. Reverse the inputted number.

Here is a program in C that takes an integer as input and prints out all the digits of that integer in reverse order.

```c
#include <stdio.h>

int main(){

    int num, n = 0;

    printf("Enter a number to reverse: ");
    scanf("%d", &num);

    while(num != 0){
        n *= 10;
        n = n + num % 10;
        num /= 10;
    }

    printf("\nReversed number: %d\n\n", n);

    return 0;
}
```

Program 14: Reversing the inputted number.



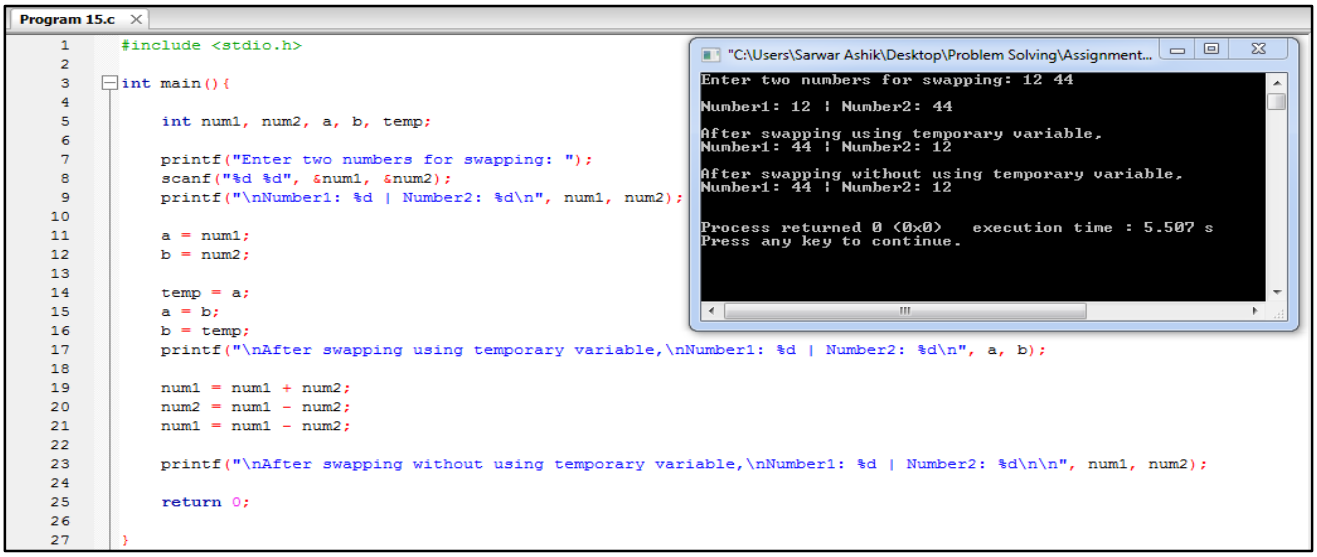
Screenshot 14: Running Program 14 in CodeBlocks and the output.

15. Swap two values with and without temporary variable.

Here is a program in C that takes two integer as input and prints out them after swapping variables.

```c
#include <stdio.h>

int main(){

    int num1, num2, a, b, temp;

    printf("Enter two numbers for swapping: ");
    scanf("%d %d", &num1, &num2);
    printf("\nNumber1: %d | Number2: %d\n", num1, num2);

    a = num1;
    b = num2;

    temp = a;
    a = b;
    b = temp;
    printf("\nAfter swapping using temporary variable,\nNumber1: %d | Number2: %d\n",
a, b);

    num1 = num1 + num2;
    num2 = num1 - num2;
    num1 = num1 - num2;

    printf("\nAfter swapping without using temporary variable,\nNumber1: %d | Number2:
%d\n\n", num1, num2);

    return 0;

}
```

Program 15: Swapping two values with and without temporary variable.



Screenshot 15: Running Program 15 in CodeBlocks and the output.

16. Multiplication of two 3*3 matrix A and B and display the result.

Here is a program in C that takes values for two matrix from standard input, multiplies them and prints out the result.

```
#include <stdio.h>
#define SIZE 3

int main(){

    int matrixA[SIZE][SIZE], matrixB[SIZE][SIZE], matrixC[SIZE][SIZE], r, c, k;

    printf("Enter the values of matrix A:\n");
    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            scanf("%d", &matrixA[r][c]);
        }
    }

    printf("\nEnter the values of matrix B:\n");
    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            scanf("%d", &matrixB[r][c]);
        }
    }

    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            matrixC[r][c] = 0;
            for(k = 0; k < SIZE; k++){
                matrixC[r][c] += matrixA[r][k] * matrixB[k][c];
            }
        }
    }

    printf("\nResult: \n");

    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            printf("%4d", matrixC[r][c]);
        }
        printf("\n");
    }

    return 0;
}
```

*Program 16: Multiplication of two 3*3 matrix A and B and display the result.*

Program 16.c

```c
#include <stdio.h>
#define SIZE 3

int main(){

    int matrixA[SIZE][SIZE], matrixB[SIZE][SIZE], matrixC[SIZE][SIZE], r, c, k;

    printf("Enter the values of matrix A:\n");
    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            scanf("%d", &matrixA[r][c]);
        }
    }

    printf("\nEnter the values of matrix B:\n");
    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            scanf("%d", &matrixB[r][c]);
        }
    }

    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            matrixC[r][c] = 0;
            for(k = 0; k < SIZE; k++){
                matrixC[r][c] += matrixA[r][k] * matrixB[k][c];
            }
        }
    }
```

Screenshot 16.1: Running Program 16 in CodeBlocks.

Program 16.c

```c
    printf("\nEnter the values of matrix B:\n");
    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            scanf("%d", &matrixB[r][c]);
        }
    }

    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            matrixC[r][c] = 0;
            for(k = 0; k < SIZE; k++){
                matrixC[r][c] += matrixA[r][k] * matrixB[k][c];
            }
        }
    }

    printf("\nResult: \n");

    for(r = 0; r < SIZE; r++){
        for(c = 0; c < SIZE; c++){
            printf("%4d", matrixC[r][c]);
        }
        printf("\n");
    }

    return 0;
}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assign...

```
Enter the values of matrix A:
1 2 3
4 5 6
7 8 9

Enter the values of matrix B:
2 3 4
5 6 7
8 9 10

Result:
  36  42  48
  81  96 111
 126 150 174

Process returned 0 (0x0)   execution time : 17.129 s
Press any key to continue.
```

Screenshot 16.2: Running Program 16 in CodeBlocks and the output.

17. Find Grade Point Average (GPA) for five subject marks. Marks should be taken into an array called "sub_marks". *[Remember Range:-80-100→ A+, Range:- 75-79→A, Range:- 70-74→A-, Range:- 65-69→B+, Range:- 60-64→B, Range:- 55-59→B-, Range:- 50-54→C+, Range:- 45-49→C, Range:- 40-44→ D and <40→Fail.]*

Here is a program in C that takes 5 values as input in an array, calculate GPA and prints out.

```
#include <stdio.h>

int main(){

    int sub_marks[5], GPA = 0, i, fail_status = 0;
    printf("Enter marks for 5 subjects -->\n");
    for(i = 0; i < 5; i++){
        input:
        printf("Subject %d: ", i + 1);
        scanf("%d", &sub_marks[i]);
        if(sub_marks[i] < 0 || sub_marks[i] > 100){
            printf("\n**Wrong input for Subject %d. Enter the mark again.\n", i + 1);
            goto input;
        }
        else if(sub_marks[i] < 40){
            fail_status = 1;
        }
    }

    if(fail_status == 1){
        printf("\nGPA: Fail\n\n");
    }
    else{

        for(i = 0; i < 5; i++){
            GPA += sub_marks[i];
        }

        GPA = GPA / 5;

        if(GPA >= 80 && GPA <= 100){
            printf("\nGPA: A+\n\n");
        }
        else if(GPA >= 75 && GPA <= 79){
            printf("\nGPA: A\n\n");
        }
        else if(GPA >= 70 && GPA <= 74){
            printf("\nGPA: A-\n\n");
        }
        else if(GPA >= 65 && GPA <= 69){
            printf("\nGPA: B+\n\n");
        }
        else if(GPA >= 60 && GPA <= 64){
            printf("\nGPA: B\n\n");
        }
        else if(GPA >= 55 && GPA <= 59){
            printf("\nGPA: B-\n\n");
        }
        else if(GPA >= 50 && GPA <= 54){
            printf("\nGPA: C+\n\n");
        }
        else if(GPA >= 45 && GPA <= 49){
            printf("\nGPA: C\n\n");
        }
        else if(GPA >= 40 && GPA <= 44){
            printf("\nGPA: D\n\n");
        }
        else if(GPA < 40){
            printf("\nGPA: Fail\n\n");
        }
        else{
            printf("\nMarks weren't correctly inputted!\n\n");
        }
    }

    return 0;
}
```

Program 17: Finding Grade Point Average (GPA) for five subject marks.

Program 17.c

```c
#include <stdio.h>

int main(){

    int sub_marks[5], GPA = 0, i, fail_status = 0;

    printf("Enter marks for 5 subjects -->\n");
    for(i = 0; i < 5; i++){
        input:
        printf("Subject %d: ", i + 1);
        scanf("%d", &sub_marks[i]);
        if(sub_marks[i] < 0 || sub_marks[i] > 100){
            printf("\n**Wrong input for Subject %d. Enter the mark again.\n", i + 1);
            goto input;
        }
        else if(sub_marks[i] < 40){
            fail_status = 1;
        }
    }

    if(fail_status == 1){
        printf("\nGPA: Fail\n\n");
    }
    else{

        for(i = 0; i < 5; i++){
            GPA += sub_marks[i];
        }

        GPA = GPA / 5;

        if(GPA >= 80 && GPA <= 100){
            printf("\nGPA: A+\n\n");
        }
        else if(GPA >= 75 && GPA <= 79){
```

Screenshot 17.1: Running Program 17 in CodeBlocks.

Program 17.c

```c
        }
        else if(GPA >= 75 && GPA <= 79){
            printf("\nGPA: A\n\n");
        }
        else if(GPA >= 70 && GPA <= 74){
            printf("\nGPA: A-\n\n");
        }
        else if(GPA >= 65 && GPA <= 69){
            printf("\nGPA: B+\n\n");
        }
        else if(GPA >= 60 && GPA <= 64){
            printf("\nGPA: B\n\n");
        }
        else if(GPA >= 55 && GPA <= 59){
            printf("\nGPA: B-\n\n");
        }
        else if(GPA >= 50 && GPA <= 54){
            printf("\nGPA: C+\n\n");
        }
        else if(GPA >= 45 && GPA <= 49){
            printf("\nGPA: C\n\n");
        }
        else if(GPA >= 40 && GPA <= 44){
            printf("\nGPA: D\n\n");
        }
        else if(GPA < 40){
            printf("\nGPA: Fail\n\n");
        }
        else{
            printf("\nMarks weren't correctly inputted!\n\n");
        }
    }

    return 0;
}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assignm...

```
Enter marks for 5 subjects -->
Subject 1: 45
Subject 2: 39
Subject 3: -5

**Wrong input for Subject 3. Enter the mark again.
Subject 3: 75
Subject 4: 78
Subject 5: 66

GPA: Fail


Process returned 0 (0x0)   execution time : 52.073 s
Press any key to continue.
```

Screenshot 17.2: Running Program 17 in CodeBlocks and the output.

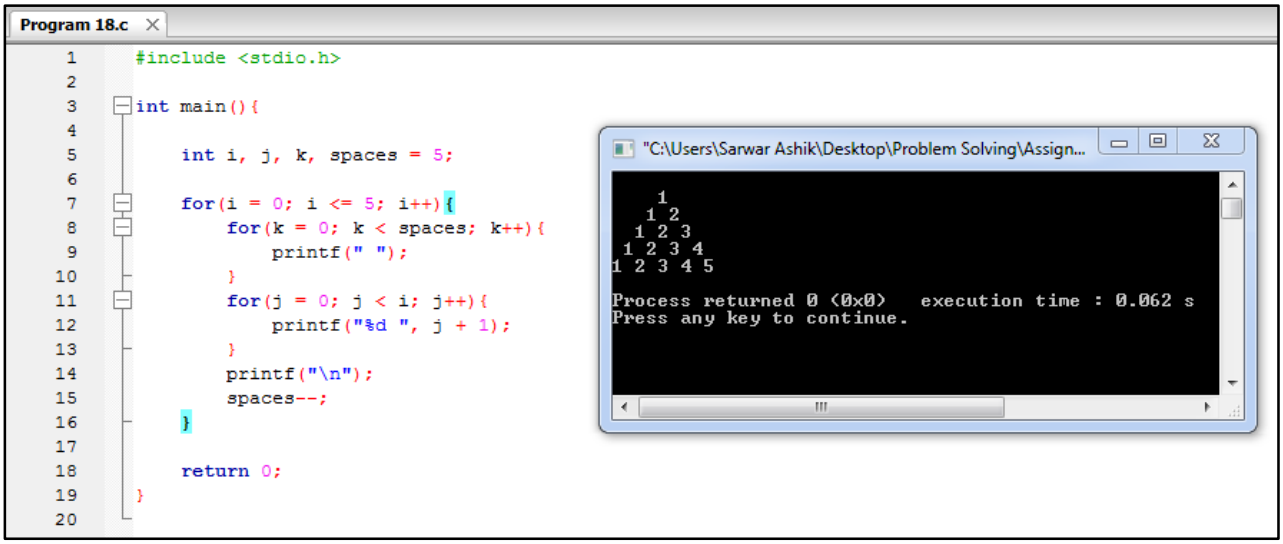
18. Generate numbers triangle like the following:-

1
12
123
1234
12345

Here is a program in C that prints out such type of number pyramid.

```c
#include <stdio.h>

int main(){

    int i, j, k, spaces = 5;

    for(i = 0; i <= 5; i++){
        for(k = 0; k < spaces; k++){
            printf(" ");
        }
        for(j = 0; j < i; j++){
            printf("%d ", j + 1);
        }
        printf("\n");
        spaces--;
    }

    return 0;
}
```

Program 18: Generating numbers triangle.



Screenshot 18: Running Program 18 in CodeBlocks and the output.

19. Search all positions of any particular value in the unsorted list called "marks".

Here is a program in C that prints out all positions of any particular value from the "marks" list.

```
#include <stdio.h>

int main(){

    int arr_len, i, j, temp, search, search_stat = 0;

    printf("How many marks for input?\nEnter a number: ");
    scanf("%d", &arr_len);

    int marks[arr_len];

    for(i = 0; i < arr_len; i++){
        printf("Mark %d: ", i + 1);
        scanf("%d", &marks[i]);
    }

    for(i = 0; i < arr_len; i++){
        for(j = i + 1; j < arr_len; j++){
            if(marks[i] > marks[j]){
                temp = marks[j];
                marks[j] = marks[i];
                marks[i] = temp;
            }
        }
    }

    printf("Enter a mark to search: ");
    scanf("%d", &search);
    printf("\nPosition(s) of %d in list: ", search);
    for(i = 0; i < arr_len; i++){
        if(search == marks[i]){
            printf("%4d", i + 1);
            search_stat = 1;
        }
    }

    if(search_stat == 0){
        printf("[NOT FOUND]\n\n");
    }
    else{
        printf("\n\n");
    }

    return 0;
}
```

Program 19: Searching all positions of any particular value in the unsorted list called "marks".

Program 19.c

```c
#include <stdio.h>

int main(){

    int arr_len, i, j, temp, search, search_stat = 0;

    printf("How many marks for input?\nEnter a number: ");
    scanf("%d", &arr_len);

    int marks[arr_len];

    for(i = 0; i < arr_len; i++){
        printf("Mark %d: ", i + 1);
        scanf("%d", &marks[i]);
    }

    for(i = 0; i < arr_len; i++){
        for(j = i + 1; j < arr_len; j++){
            if(marks[i] > marks[j]){
                temp = marks[j];
                marks[j] = marks[i];
                marks[i] = temp;
            }
        }
    }
```

Screenshot 19.1: Running Program 19 in CodeBlocks.

Program 19.c

```c
                marks[i] = temp;
            }
        }
    }

    printf("Enter a mark to search: ");
    scanf("%d", &search);
    printf("\nPosition(s) of %d in list: ", search);
    for(i = 0; i < arr_len; i++){
        if(search == marks[i]){
            printf("%4d", i + 1);
            search_stat = 1;
        }
    }

    if(search_stat == 0){
        printf("[NOT FOUND]\n\n");
    }
    else{
        printf("\n\n");
    }

    return 0;
}
```

"C:\Users\Sarwar Ashik\Desktop\Problem Solving\Assignment...

```
How many marks for input?
Enter a number: 5
Mark 1: 11
Mark 2: 12
Mark 3: 11
Mark 4: 33
Mark 5: 44
Enter a mark to search: 11

Position(s) of 11 in list:    1   2


Process returned 0 (0x0)   execution time : 22.138 s
Press any key to continue.
```

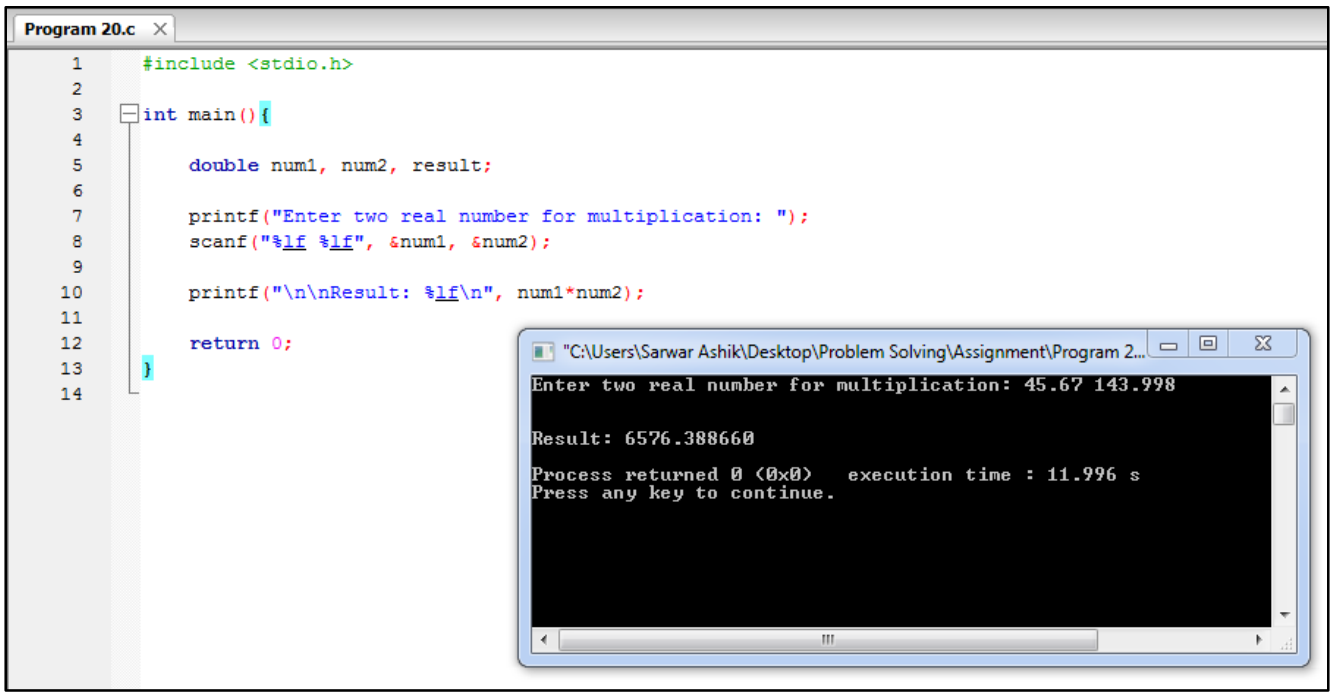
Screenshot 19.2: Running Program 19 in CodeBlocks and the output.

20. Multiply two float type numbers.

Here is a program in C that takes 2 float type numbers as input, multiplies them and prints out the result.

```c
#include <stdio.h>

int main(){

    double num1, num2, result;

    printf("Enter two real number for multiplication: ");
    scanf("%lf %lf", &num1, &num2);

    printf("\n\nResult: %lf\n", num1*num2);

    return 0;
}
```

Program 20: Multiplying two float type numbers.



Screenshot 20: Running Program 20 in CodeBlocks and the output.